

SpecForge: A Flexible and Efficient Open-Source Training Framework for Speculative Decoding

Shenggui Li^{1,2} Chao Wang^{3,2} Yikai Zhu² Yubo Wang² Fan Yin² Shuai Shi² Yefei Chen²
 Xiaomin Dong⁴ Qiaoling Chen¹ Jin Pan² Ji Li² Laixin Xie² Yineng Zhang² Lei Yu³ Yonggang Wen¹
 Ivor Tsang⁵ Tianwei Zhang¹

GitHub: <https://github.com/sgl-project/SpecForge>

Abstract

Large language models (LLMs) incur high inference latency due to sequential autoregressive decoding. Speculative decoding alleviates this bottleneck by using a lightweight draft model to propose multiple tokens for batched verification. However, its adoption has been limited by the lack of high-quality draft models and scalable training infrastructure. We introduce SpecForge, an open-source, production-oriented framework for training speculative decoding models with full support for EAGLE-3. SpecForge incorporates target-draft decoupling, hybrid parallelism, optimized training kernels, and integration with production-grade inference engines, enabling up to $9.9\times$ faster EAGLE-3 training for Qwen3-235B-A22B. In addition, we release SpecBundle, a suite of production-grade EAGLE-3 draft models trained with SpecForge for mainstream open-source LLMs. Through a systematic study of speculative decoding training recipes, SpecBundle addresses the scarcity of high-quality drafts in the community, and our draft models achieve up to $4.48\times$ end-to-end inference speedup on SGLang, establishing SpecForge as a practical foundation for real-world speculative decoding deployment.

1. Introduction

Large language models (LLMs) have rapidly become a cornerstone of modern AI systems. Both proprietary models—such as ChatGPT (Achiam et al., 2023), Gemini (Comanici et al., 2024), and Grok—and open-source

counterparts, including LLaMA (Dubey et al., 2023), DeepSeek (DeepSeek-AI et al., 2024; 2025), and Qwen (Bai et al., 2023; Qwen et al., 2025; Yang et al., 2025), have driven substantial productivity gains across a wide range of industries. However, as model sizes continue to scale, inference latency has emerged as a fundamental bottleneck (Yu & Jeong, 2022; Recasens et al., 2025). LLMs’ autoregressive generation requires a full forward pass through billions of parameters for each token, resulting in a memory-bound inference process that significantly increases deployment cost and hinders real-time or high-throughput applications.

Speculative decoding has emerged as a promising remedy, offering substantial speedups by pairing a small draft model with a large target model (e.g., the original model) (Leviathan et al., 2022; Chen et al., 2023). The draft model quickly generates several candidate tokens, and the target model then verifies multiple tokens in parallel via a single forward pass. The draft model can be in the form of N-Gram models (Fu et al., 2024), models of smaller size from the same model family (Leviathan et al., 2022; Chen et al., 2023), sub-layers of the same model (Zhang et al., 2024a; Liu et al., 2024; Xia et al., 2024), and additional autoregressive adapters (Cai et al., 2024; Li et al., 2024b;a; Zhang et al., 2024b; Du et al., 2024; Li et al., 2025). If the draft’s predictions are likely to be correct as judged by the target model, they are accepted; otherwise, the target model corrects them. The number of forward passes of the target model is thus significantly reduced. As a result, by effectively leveraging extra parallel computation when available, speculative decoding can reduce inference time substantially without changing the distribution of outputs.

Early demonstrations of speculative decoding (Leviathan et al., 2022) showed speedups of up to $3.4\times$ on large Transformer models such as T5-XXL (Raffel et al., 2020), while provably preserving output fidelity. Subsequent advances have further improved its efficiency and practicality. Notably, the EAGLE-3 algorithm (Li et al., 2025) introduces a draft model that operates at a hybrid feature level, substantially increasing token acceptance rates and achieving

¹Nanyang Technological University ²SpecForge Team
³Meituan ⁴EigenAI ⁵A*STAR, Singapore. Correspondence to:
 Tianwei Zhang <tianwei.zhang@ntu.edu.sg>.

up to 4.79 $\times$ speedup on LLaMA-3.3-70B without quality degradation. EAGLE-3 further incorporates **dynamic tree-based generation and a Training-Time Test (TTT) procedure that better simulates multi-step decoding during draft training**. Owing to its strong empirical performance, EAGLE-3 has become the de facto industrial standard for speculative decoding and is supported by major inference engines, including open-source systems such as SGLang (Zheng et al., 2023) and vLLM (Kwon et al., 2023), as well as proprietary platforms like TensorRT-LLM (trt, 2025).

However, despite the strong theoretical guarantees and empirical gains, speculative decoding remains underutilized in practice, especially for EAGLE3. We identify three main causes which hinder the wider application of speculative decoding in inference:

Cause 1: Limited availability of draft models. The effectiveness of speculative decoding critically depends on the adoption of a well-trained draft model that **closely approximates the predictions of the target model**. Such a draft model is often unavailable in practice. Early approaches (Leviathan et al., 2022) assume the existence of smaller models from the same family as the target model, which frequently does not hold. For example, Kimi K2 (Team et al., 2025), a 1-trillion-parameter model, was **released without a corresponding smaller variant, rendering this approach infeasible**. Even with state-of-the-art methods such as EAGLE-3, many mainstream models, including Qwen3, lack publicly available matching draft models, significantly hindering the practical adoption of speculative decoding.

Cause 2: Poor performance of open-source draft models. Previous work (Li et al., 2025; 2024b) has released a **limited number of draft-model weights on Hugging Face**, enabling engineers and researchers to directly integrate them into inference frameworks such as SGLang, vLLM, and TensorRT-LLM for acceleration. However, these publicly available drafts are typically **trained on relatively small, research-oriented datasets, which limits their robustness and renders them unsuitable for production-level deployment**.

To quantify this limitation, we reproduced the EAGLE-3 training procedure for LLaMA-3.1-Instruct using ShareGPT (120K conversations) and UltraChat (200K conversations) (Ding et al., 2023). Under this setting, the resulting draft model achieved an acceptance length of 2.82 on the Math500 benchmark. In contrast, training the same draft model on the Perfect-Blend dataset, which contains 1.4M conversations, improved the acceptance length to 3.48, corresponding to an additional 1.17 $\times$ inference speedup.

This gap highlights a broader mismatch in the open-source LLM ecosystem: while foundation models such as DeepSeek, Qwen, and GLM have rapidly advanced to state-of-the-art performance, high-quality draft models remain

scarce and underdeveloped, leaving substantial headroom for improving the effectiveness of speculative decoding.

Cause 3: Lack of robust training tools. Constructing a high-quality draft model is inherently non-trivial, often requiring architectural customization and the implementation of **sophisticated training mechanisms such as Training-Time Test (TTT)** (Li et al., 2025). Until recently, practitioners lacked robust tooling to support this process. Most existing speculative decoding implementations remain ad hoc, fragmented, or ill-suited for large-scale training (eag, 2025). Given that target models can range from several billion to over one trillion parameters, any practical training framework must be highly scalable, efficient, and reliable. The absence of such infrastructure has substantially hindered the community’s ability to train high-quality draft models, thereby limiting their availability and adoption across real-world and open-source ecosystems.

SpecForge is our attempt to fill these gaps by advancing the practical development of speculative decoding in both research and industry. It is a unified, production-oriented framework for training draft models for speculative decoding, offering native support for advanced algorithms such as EAGLE-3, including the complex Training-Time Test (TTT) procedure with tree attention masks and recursive scheduling. With SpecForge, practitioners can easily train state-of-the-art draft models through simple configuration rather than custom engineering.

To ensure that these capabilities operate at scale, SpecForge adopts a **hybrid parallelism strategy via target-draft decoupling**, which **explicitly decouples the target model and the draft model, enabling each to be parallelized according to its distinct computational characteristics**. In speculative decoding training, the target model is typically large, frozen, and inference-dominated, while the draft model is lightweight and frequently updated. Treating both models as a single monolithic modules, as done in prior implementation, forces a uniform parallelization strategy that is suboptimal for both. SpecForge instead applies **inference-oriented parallelism to the target model leveraging the integration with SGLang and training-oriented strategies to the draft model**. This separation improves scalability, reduces communication overhead, and allows the framework to efficiently support target models ranging from billions to over a trillion parameters.

SpecForge further optimizes the Training-Time Test (TTT) procedure in EAGLE-3 by introducing **memory- and compute-efficient attention implementations tailored to its autoregressive multi-step structure**. It leverages the **sparsity pattern in tree attention** to reduce the computation time and memory peak, and optimizes the loss computation via **customized in-place operations**. Together, these optimizations significantly lower memory consumption and wall-clock

time, enabling stable and scalable EAGLE-3 training at long context lengths.

To enrich the availability of high-quality draft models in the open ecosystem, we have trained a comprehensive suite of draft models, named SpecBundle, covering mainstream open-source LLM families including Llama-3, Llama-4, Qwen-3, GPT-OSS, Kimi K2, and DeepSeek V3. SpecBundle is built on extensive, diverse training corpora specifically for speculative decoding, offering substantially stronger draft quality than existing open-source checkpoints. In empirical evaluations, SpecBundle models deliver up to 4.8x speedup over inference without speculative decoding and 1.3x speedup over publicly available draft model checkpoints across multiple task domains, making them practical drop-in draft models for real-world deployment.

During the training of SpecBundle, we also systematically investigated the properties of speculative decoding and derived practical training recipes that elucidate key design choices, including draft model architectures, dataset quality, and the configuration of training-time test. Together, these findings provide actionable guidance for building high-quality draft models and inform best practices for deploying speculative decoding in production settings.

We summarize our main contributions and key novel features of SpecForge as follows:

- We introduce SpecForge, an efficient and scalable training framework for speculative decoding. SpecForge implements hybrid parallelism through target-draft decoupling and training-time test attention optimization, enabling large-scale and efficient draft model training.
- We release SpecBundle, a collection of high-quality draft models covering mainstream open-source LLM families. It delivers stronger accuracy and up to 1.3x speedup over existing open checkpoints, addressing the limited availability of draft models in the open-source ecosystem.
- We systematically investigate training recipes and design choices for improving speculative decoding performance, providing practical insights for real-world deployment and directions for future research.

2. Preliminaries

2.1. Speculative Decoding

Speculative decoding (Chen et al., 2023; Leviathan et al., 2022; Xia et al., 2023; Miao et al., 2024; Cai et al., 2024; Li et al., 2024b;a; 2025; Hu et al., 2025) has emerged as the premier algorithmic intervention to address this memory-bound inefficiency without requiring the retraining of the foundational model or compromising generation quality. First formalized effectively by Leviathan et al. (2022), it fundamentally restructures the inference workload. It replaces the serial, memory-intensive generation of single tokens

with a parallel, compute-intensive verification of candidate sequences. By employing a computationally inexpensive “draft model” to propose short sequences of tokens (speculation), speculative decoding allows the massive “target model” to verify these proposals in a single forward pass. This effectively converts the sequential generation problem into a batch processing problem, thereby increasing arithmetic intensity and better utilizing the massive parallel compute capabilities of modern hardware.

2.2. Theoretical Speedup

The efficiency of speculative decoding is governed by the trade-off between the time saved by accepting draft tokens and the overhead of generating them. The number of tokens generated per single run of the target model is a random variable dependent on the quality of the draft model. Assuming the acceptance of each token is an independent event with probability α (the acceptance rate), the expected number of tokens generated per cycle is derived from a truncated geometric distribution:

$$E[\text{tokens}] = \frac{1 - \alpha^{\gamma+1}}{1 - \alpha}$$

where γ is the number of speculative draft tokens. As α approaches 1 (perfect alignment between draft and target), the expected length approaches $\gamma + 1$. The theoretical walltime speedup S is defined as the ratio of standard autoregressive time to speculative decoding time. Let c be the cost ratio between the draft model and the target model ($c = C_d/C_p$). The speedup is given by:

$$S = \frac{E[\text{tokens}]}{1 + \gamma c} = \frac{1 - \alpha^{\gamma+1}}{(1 - \alpha)(1 + \gamma c)}$$

This equation reveals the critical efficiency bounds:

- Draft Quality (α): Maximizing α is paramount. The acceptance rate is fundamentally limited by the Kullback-Leibler (KL) divergence between the draft and target distributions.
- Draft Cost (c): The draft model must be significantly cheaper than the target ($c \ll 1$). If the draft overhead γc becomes too large, it negates the parallelization gains.
- Speculation Length (γ): There is an optimal γ for any given α and c . While increasing γ raises the potential tokens per step, it linearly increases the draft overhead. Modern frameworks often tune γ dynamically.

2.3. Architecture Paradigm

The draft model in speculative decoding has undergone a series of paradigm shift.

Stage 1: Independent Draft Model. The earliest realizations of speculative decoding adopted a straightforward architectural strategy in which a smaller, independently

trained language model serves as the draft model (Leviathan et al., 2022; Chen et al., 2023; Sun et al., 2023; Kim et al., 2023). While conceptually simple, this paradigm imposes significant system-level constraints. To preserve tokenizer compatibility, a large target model (e.g., Chinchilla-70B or LLaMA-2-70B) must be paired with a substantially smaller model, often from the same family (e.g., 7B variants). This tight architectural coupling introduces an inherent alignment gap: smaller models tend to produce probability distributions that diverge from those of their larger counterparts, particularly on complex reasoning or long-context tasks, resulting in elevated token rejection rates. From the system perspective, maintaining two independent models also increases memory pressure, as both model parameters and KV caches must reside in GPU memory. In distributed settings, this design further incurs synchronization and communication overhead, which can erode the theoretical speedups of speculative decoding.

Stage 2: Multi-Token Prediction Heads. Medusa (Cai et al., 2024) introduced multiple prediction heads to eliminate the overhead of independent draft models and dual KV-caches. It adds lightweight heads that run in parallel with the standard LM head, enabling zero-latency drafting: generating candidate tokens costs nearly the same walltime as generating one because the additional MLPs are small and executed concurrently with the backbone. These heads reuse the target model’s features, avoiding a separate KV-cache and ensuring strong alignment. Verification is performed in a single forward pass using Tree Attention (Cai et al., 2024), where a masked attention structure constrains each candidate token to attend only to its ancestors, allowing the model to evaluate multiple hypotheses in parallel and preserve useful branches even when others are rejected.

Stage 3: Feature-Level Extrapolation. Although Medusa eliminates the overhead of maintaining an independent draft model, its non-autoregressive MLP heads struggle to capture long-range dependencies. Li et al. (2024b) address this limitation with EAGLE, which shifts autoregression from token space to feature space under the feature-uncertainty hypothesis: hidden-state trajectories in high-dimensional feature space are smoother and more predictable than the discrete jumps between tokens (Li et al., 2024b; Du et al., 2024). EAGLE replaces the standalone draft model with a lightweight single-layer Transformer that autoregressively predicts future feature representations, which are then projected through a linear layer to obtain the token logits. This fully autoregressive yet efficient design enables accurate multi-step drafting and achieves substantial empirical speedups. Its successor, EAGLE-2 (Li et al., 2024a), further improves performance by dynamically shaping the draft tree according to token-level confidence, allocating verification compute to the most promising candidates.

Algorithm 1 TTT Attention

Input: query q_t , prefix keys K^{train} , prefix values V^{train} , cached keys $\{k_i\}_{i>T}$, cached values $\{v_i\}_{i>T}$
Output: attention output o_t

```

 $S \leftarrow \frac{q_t (K^{\text{train}})^\top}{\sqrt{d_k}}$ 
for  $i \leftarrow T + 1$  to  $t - 1$  do
   $s_i \leftarrow \frac{q_t \cdot k_i}{\sqrt{d_k}}$ 
   $S \leftarrow \text{concat}(S, s_i)$ 
end for
 $\alpha \leftarrow \text{softmax}(S)$ 
 $o_t \leftarrow \alpha \cdot V^{\text{train}}$ 
for  $i \leftarrow T + 1$  to  $t - 1$  do
   $o_t \leftarrow o_t + \alpha_i v_i$ 
end for

```

2.4. Training-Time Test

In the latest upgrade, EAGLE-3 (Li et al., 2025) additionally incorporates *Training-Time Testing* (TTT) to autoregressively generate the next few tokens, reducing error accumulation and improving acceptance rates in multi-token prediction. The core idea of TTT is to simulate multiple steps of autoregressive token generation during training. As shown in Algorithm 1, at each TTT step, the model attends to a growing context consisting of the original training sequence as prefix and the tokens generated in previous steps.

Let T denote the length of the training prefix. For any position $t > T$, positions $1:T$ correspond to the training sequence, while positions $T+1:t-1$ correspond to representations predicted during earlier TTT steps. At position t , the model computes attention using the query vector q_t over keys and values concatenated.

We define the key and value matrices for the training prefix and previously predicted tokens as

$$K^{\text{train}} = [k_1, \dots, k_T], \quad K^{\text{pred}} = [k_{T+1}, \dots, k_{t-1}], \\ V^{\text{train}} = [v_1, \dots, v_T], \quad V^{\text{pred}} = [v_{T+1}, \dots, v_{t-1}].$$

The attention output at step t is computed as

$$o_t = \text{softmax} \left(\frac{q_t \begin{bmatrix} K^{\text{train}} \\ K^{\text{pred}} \end{bmatrix}^\top}{\sqrt{d_k}} \right) \begin{bmatrix} V^{\text{train}} \\ V^{\text{pred}} \end{bmatrix},$$

where d_k denotes the key dimensionality.

Equivalently, the attention logits can be decomposed into prefix and prediction components as

$$S_t = \left[\frac{q_t K^{\text{train}\top}}{\sqrt{d_k}}, \frac{q_t K^{\text{pred}\top}}{\sqrt{d_k}} \right].$$

Intuitively, the attention logits S_t decompose into two parts: (i) causal attention between the query q_t and the training prefix keys K^{train} , and (ii) dot products between q_t and the keys k_i generated in previous TTT steps for $i > T$:

3. Challenges

Despite the rapid growth of speculative decoding, especially EAGLE3, training the draft model has received less attention. Compared to large-scale model training using frameworks like Megatron (Narayanan et al., 2021) and DeepSpeed (Rasley et al., 2020), one significant attribute of EAGLE3 training is that the number of trainable parameters is smaller by a magnitude as the EAGLE3 draft model is often one-layer Transformer. Nonetheless, constructing a draft model is non-trivial because of the following challenges.

Rigid Parallelism Strategies. Existing open-source implementations (eag, 2025; mod, 2025) treat the target and draft model as a unified model, and apply fully sharded data parallelism (FSDP) (Zhao et al., 2023; Rasley et al., 2020) for training by wrapping both models. Despite its simplicity and user-friendliness, such unified parallelism strategy is sub-optimal for several reasons. First, even though the draft models are typically small, the target models can vary from several billions of parameters to trillions of parameters. ZeRO-style sharding (Rasley et al., 2020) is not optimal for all scales and thus does not provide high-performance hidden-state generation. It is evident that current high-performance generation engines like SGLang, vLLM and TensorRT favour tensor parallelism and expert parallelism over all-gather-based ZeRO-style sharding. Consequently, treating the target and draft models as a single monolithic module limits both performance and scalability.

Sub-optimal Prefill Performance. The training process of EAGLE3 can be naturally decomposed into two stages. In the first stage, the target model is executed over the entire input sequence to generate the corresponding hidden states. This is equivalent to the prefill phase in standard LLM inference, where the model processes the tokens in a fully autoregressive manner in parallel before decoding begins.

However, existing EAGLE3 training frameworks typically rely on naïve model implementations, either self-written or directly imported from Hugging Face. These implementations are primarily designed for general-purpose training and correctness, rather than for high-throughput inference workloads. As a result, they fail to exploit many inference-specific optimizations that have been extensively engineered into mature, production-grade inference engines. In particular, these training pipelines lack optimizations such as efficient attention kernels, optimized memory management, and CUDA Graph of which are critical for accelerating the prefill stage. In contrast, modern inference engines like

SGLang and vLLM are explicitly optimized for this execution pattern and can deliver substantially higher throughput and better hardware utilization during prefill.

This mismatch leads to a significant inefficiency in training: the prefill stage often becomes a dominant bottleneck in large-scale draft-model training, inflating both training time and resource consumption. Addressing this gap requires rethinking the training pipeline to better align with inference-optimized execution.

4. SpecForge

We propose several techniques to tackle the above challenges and optimize the overall training performance.

4.1. Target-Draft Decoupling

The original EAGLE3 design tightly couples the draft and target models into a single parallelized module, as illustrated in Figure 2a. While this design simplifies implementation, it is suboptimal from a performance standpoint. More critically, training and inference engines are optimized for fundamentally different objectives and system constraints; tightly coupling the two models prevents the simultaneous use of state-of-the-art training frameworks and high-performance inference engines. Decoupling the draft and target models therefore emerges as a key abstraction for achieving scalability, efficiency, and deployment flexibility.

For the draft model, the primary challenge lies in training efficiency which has been well supported by mature training frameworks such as DeepSpeed (Rasley et al., 2020) and Megatron (Narayanan et al., 2021), offering extensive parallelization for distributed training. In contrast, the target model is typically large and inference-only, making it better suited to specialized inference engines such as SGLang (Zheng et al., 2023). By decoupling the two models and applying distinct execution backends and parallelization strategies, SpecForge enables each component to operate in its optimal regime. This also allows the direct deployment of trained draft models on optimized inference engines, resulting in a seamless and production-ready workflow.

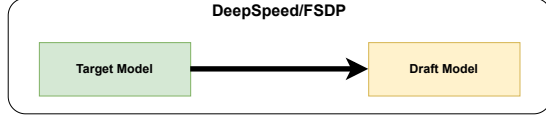
4.1.1. HYBRID PARALLELISM

To accommodate the distinct characteristics of target and draft models, we leverage the SGLang inference engine and Fully Sharded Data Parallelism for them respectively, as shown in Figure 2b.

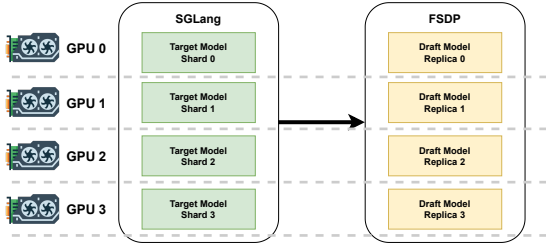
For the draft model, this design choice is motivated by two key observations. First, the draft model is typically only 3–5% the size of the target model, rendering heavy-weight parallelization schemes such as tensor or pipeline parallelism unnecessary or even counterproductive. Second,

query \ key	Prefix Sequence				Step 1				Step 2			
	How	can	...	<pad>	are	we	...	<pad>	you	help	...	<pad>
you	1	0	0	0	1	0	0	0	1	0	0	0
help	1	1	0	0	0	1	0	0	0	1	0	0
...	1	1	1	0	0	0	1	0	0	0	1	0
<pad>	0	0	0	0	0	0	0	0	0	0	0	0

Figure 1. EAGLE3 attention mask used in Training-Time Testing.



(a) Existing implementation wraps both the target model and draft model into a single parallel strategy



(b) SpecForge decouples the target model and draft model with hybrid parallelism

Figure 2. Architecture comparisons

training and inference stages impose fundamentally different compute and memory requirements, making specialized, stage-specific optimizations critical to overall system performance. Given the relatively small size of the draft model, we only shard the optimizer states and gradients to minimize the communication overhead, which is equivalent to ZeRO Stage 2 in DeepSpeed (Rasley et al., 2020).

For the target model, we directly employ the model runner provided by SGLang as the inference engine. This allows us to reuse SGLang’s existing parallelization strategies—including tensor parallelism, expert parallelism, and pipeline parallelism—as well as high-performance kernels such as FlashAttention (Shah et al., 2024) and FlashInfer (Ye et al., 2025) to accelerate the prefill phase of target-model inference. In addition, we can apply piecewise CUDA Graph in SGLang to fuse non-attention modules into a single kernel to reduce kernel launch time.

By decoupling the parallel strategies of the target model and draft model, they can be either co-located on the same GPU, or disaggregated on distinct GPUs. For our experiments, we conducted evaluation under the co-locate settings.

Algorithm 2 BlockMask Construction for Training-Time Testing

Input: batch index b , query index q_i , key/value index kv_i , prefix length Q_{LEN} , sequence length T

Output: BlockMask M

// Causal mask

$$m_{\text{causal}} \leftarrow (q_i \geq kv_i)$$

$$m_{\text{pad}} \leftarrow (kv_i < T)$$

$$M_{\text{causal}} \leftarrow m_{\text{causal}} \wedge m_{\text{pad}}$$

// Suffix mask

$$m_{\text{suffix}} \leftarrow (kv_i \geq Q_{LEN})$$

$$m_{\text{pad}} \leftarrow (kv_i \bmod Q_{LEN} < T)$$

$$m_{\text{diag}} \leftarrow ((kv_i - q_i) \bmod Q_{LEN} = 0)$$

$$M_{\text{suffix}} \leftarrow m_{\text{suffix}} \wedge m_{\text{pad}} \wedge m_{\text{diag}}$$

// BlockMask

$$M \leftarrow M_{\text{causal}} \vee M_{\text{suffix}}$$

4.2. Computation Optimization

Beyond parallelization strategies, we further investigated the training characteristics of the draft model and observed that Training-Time Test (TTT) with a step length of 7 incurs substantial GPU memory consumption. To address this bottleneck, we design two complementary optimizations that significantly reduce memory usage during training.

4.2.1. SPARSE TREE ATTENTION

The naive implementation in Algorithm 1 materializes the attention logits S as intermediate activations. As TTT runs multiple autoregressive steps in the forward pass, these logits accumulate and quickly dominate memory usage. Our profiling shows that stored attention logits account for **80%** of the total activation memory, making them the primary memory bottleneck during training.

To reduce the memory footprint, we leverage *FlexAttention* to compute attention. FlexAttention is a PyTorch project that leverages TorchInductor to compile a Python DSL into

a Triton kernel. It brings two benefits: (1) FlexAttention computes attention in a FlashAttention-style streaming manner, avoiding the need to save intermediate activations; and (2) FlexAttention implements a *BlockMask* data structure, which efficiently precomputes blocks that can be skipped, partially computed, or fully computed, and optimizes the implementation accordingly. To use *FlexAttention*, we construct a *BlockMask* that encodes the allowed attention blocks, as illustrated in Figure 1 and Algorithm 2. At each TTT step we store the newly generated keys and values in the KV cache and construct a custom attention mask represented as a *BlockMask*, which is then provided to the FlexAttention operator during attention computation.

4.2.2. MEMORY-EFFICIENT GRADIENT COMPUTATION

To further reduce memory usage, we implement the backward pass of the masked softmax loss with a custom Triton kernel (Algorithm 3). The key idea is to reuse the input logits tensor to store gradients during the backward pass. After the forward loss computation, the logits are no longer needed. Instead of allocating a separate gradient buffer, the Triton kernel overwrites the logits tensor with the gradient with respect to logits. This avoids storing additional activation or gradient tensors and reduces memory overhead. The memory reduction ranges from **30–40%**, depending on the context length and the draft model’s vocabulary size.

Algorithm 3 In-place Backward for Log-Softmax Loss

Input: logits z , target p , upstream gradient g

Output: gradient w.r.t. logits (stored in z)

$s \leftarrow \sum(p \cdot g)$

$\pi \leftarrow \text{softmax}(z)$

$z \leftarrow -(p \cdot g - \pi \cdot s)$

5. Evaluation of SpecForge

5.1. Experimental Setup

We evaluated the system performance of SpecForge and compare it against existing implementations. We considered two publicly available codebases: (1) the official implementation released by SafeAILab alongside the EAGLE3 paper, and (2) a third-party implementation developed by NVIDIA’s Model Optimizer team. As both implementations adopt a similar monolithic architecture wrapping both the target and draft models within DeepSpeed, we select the official SafeAILab implementation as our baseline. All experiments were conducted on a cluster of eight NVIDIA H200 GPUs with a sequence length of 4096. The batch size was adjusted for each method to maximize throughput under GPU memory constraints.

5.2. End-to-end Performance

We conducted end-to-end training experiments on four models spanning different scales and architectures: LLaMA3.1-8B, LLaMA3.3-70B, Qwen3-30B-A3B, and Qwen3-235B-A22B, and measured training throughput in tokens per second. For SpecForge, we enabled tensor parallelism (TP), FlashAttention kernels, and CUDA Graphs, with the tensor parallel size chosen according to the scale of the target model. The draft model was trained using ZeRO Stage 2 to achieve memory-efficient data-parallel execution. For the baseline, we evaluated both ZeRO Stage 2 and ZeRO Stage 3 configurations and report the best-performing setting.

Table 1 summarizes the results. SpecForge consistently outperforms the baseline across all model scales, achieving a maximum speedup of 9.99 $\times$. The poor performance of the baseline can be attributed to two primary factors:

- Under ZeRO Stage 2, although gradients and optimizer states are sharded, the frozen target model parameters remain fully replicated on each device, leading to rapid scalability degradation as model size increases.
- ZeRO Stage 3 shards parameters, optimizer states, and gradients; however, frequent all-gather operations during target-model inference introduce substantial communication overhead, which severely limits throughput.

These results further highlight the effectiveness of target-draft decoupling. For large-scale models such as Qwen3-235B-A22B, ZeRO-style sharding leads to extremely low throughput due to communication-dominated execution. In contrast, SpecForge consistently achieves strong performance by avoiding unnecessary synchronization and applying model-specific parallelization strategies. The results also underscore the importance of integrating with a mature inference engine like SGLang. As shown by the LLaMA-3.1-8B experiments, even when neither the baseline nor SpecForge parallelizes the target model, SpecForge still attains a 2.01 $\times$ speedup, owing to the highly optimized prefill execution provided by SGLang.

5.3. Impact of Target Model Backends

In addition, we investigated the impact of different target model backends on training performance. In SpecForge, we have supported three types of execution backends:

- **Hugging Face Backend:** Reuses model implementations from Hugging Face Transformers and relies on its internal *tp_plan* for tensor parallelism, when available.
- **SGLang Backend:** Reuses model implementations provided by SGLang and leverages its system-level optimizations, including chunked prefill (Agrawal et al., 2025), `torch.compile`, CUDA Graphs, and high-performance kernels (Ye et al., 2025; Shah et al., 2024).
- **Custom Backend:** Includes models manually implemented by our team. This backend is particularly useful

Target Model	Framework	Target Model	Draft Model	Max Batch Size	Seq Length	Step Time (s)	Throughput (tokens/s)	speedup
Llama3.1-8B	EAGLE	ZeRO 2	ZeRO 2	16	4096	1.04	63015.4	1
	SpecForge	TP=1	ZeRO 2	64		2.07	126639.6	2.01
Llama3.3-70B	EAGLE	ZeRO 2	ZeRO 2	16	4096	OOM	-	-
	EAGLE	ZeRO 3	ZeRO 3	8		2.21	14827.1	1
	SpecForge	TP=4	ZeRO 2	16		3.18	20608.8	1.39
Qwen3-30B-A3B	EAGLE	ZeRO 2	ZeRO 2	8	4096	1.12	29257.1	1
	EAGLE	ZeRO 3	ZeRO 3	8		5.07	6463.1	0.2
	SpecForge	TP=4	ZeRO 2	16		0.52	126030.8	4.31
Qwen3-235B-A22B	EAGLE	ZeRO 2	ZeRO 2	8	4096	OOM	-	-
	EAGLE	ZeRO 3	ZeRO 3	8		11.2	2025.7	1
	SpecForge	TP=8	ZeRO 2	8		1.62	20227.2	9.99

Table 1. End-to-end performance on various models.

when a model is unavailable in Hugging Face Transformers or SGLang, or when the Hugging Face implementation lacks built-in parallelization support.

We conducted training experiments on the same set of models and tensor parallel configurations in Table 1. As shown in Figure 3, SGLang significantly outperforms the other two execution backends, achieving speedups of up to 6.8 $\times$. These results highlight a key observation: optimizing the prefill stage is non-trivial, particularly for MoE models. For the Qwen3 experiments, both our custom backend and the Hugging Face backend exhibit substantially lower training throughput compared to SGLang. Notably, the Hugging Face implementation encounters runtime errors and fails to robustly support large-scale MoE models, further underscoring the importance of integrating with a mature, inference-optimized backend for scalable EAGLE3 training.

Another engineering advantage of integrating with SGLang is the clear separation of responsibilities. Model support and low-level inference optimizations can be delegated to the engine team, which typically adds support for newly released models promptly. This allows SpecForge to focus on training-specific optimizations and system design, rather than duplicating model integration and maintenance efforts.

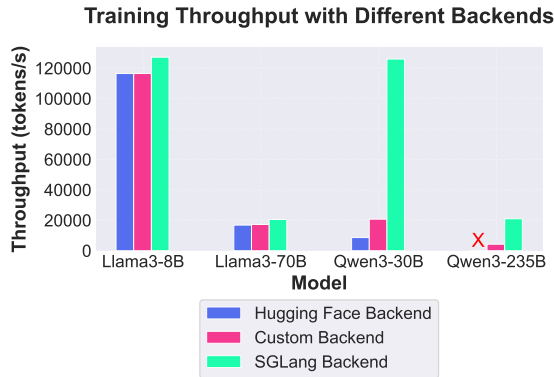


Figure 3. Training time with different execution backends

5.4. Impact of Optimization Attention Kernel

To evaluate the performance gains from our optimized attention kernel, we conducted micro-benchmarks comparing its execution time and peak memory usage against a native SDPA-based implementation. We set the TTT length to 7 and report measurements from the final TTT step. As shown in Figure 4, our optimized attention substantially reduces both wall-clock time and memory consumption. For a sequence length of 4096, it achieves reductions of 62.1% in execution time and 93.5% in peak memory usage on a single NVIDIA H200 GPU. Moreover, the performance gap widens as the sequence length increases, highlighting the effectiveness of our optimized kernel for training EAGLE3 under long-context settings.

6. SpecBundle

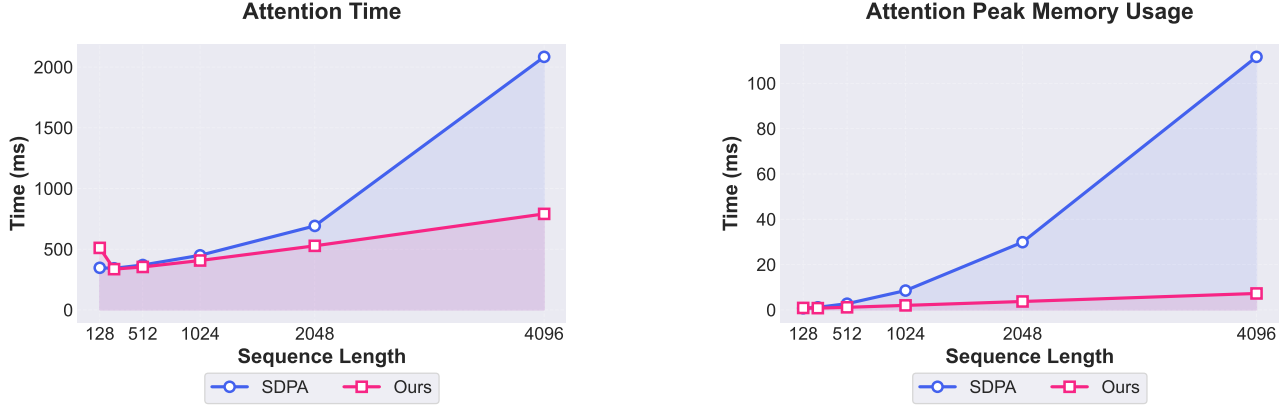
As part of our open-source efforts, we trained the EAGLE3 draft models for a collection of mainstream open-source models including Llama, Qwen and Kimi. This collection is named SpecBundle.

We trained these on models on the Open-PerfectBlend dataset (Xu et al., 2024), which consists of offers balanced 1.4M conversation in the chat, math, coding, instruction following domains. To achieve the best performance, we regenerated the assistant’s responses in the dataset using the target model with temperature 0.8 and trained the model from scratch on the regenerated dataset. We trained the draft models for 2 epochs at learning rate 1e-4 with cosine annealing scheduler.

6.1. Evaluation Results

We evaluated the results of SpecBundle on a wide range of benchmark datasets:

1. Instruction-following: MTBench (Chen et al., 2025)
2. Math: Math500 and GSM8K (Zhang & Math-AI, 2024)
3. Coding: HumanEval (Chen et al., 2021) and LCB (Jain et al., 2024)
4. Other Subjects: GPQA (Rein et al., 2024) and Fi-



(a) Kernel wall time for attention

(b) Peak memory consumption for attention

Figure 4. Comparison of execution time and memory usage between naive EAGLE3 attention and our optimized kernel.

Target Model	Draft Model	#GPUs	MTBench		GPQA		FinanceQA	
			Throughput	Speedup	Throughput	Speedup	Throughput	Speedup
Llama-3.1-8B	-	1	190.0	1	190.5	1	185.7	1
	Existing		454.7	2.39	438.1	2.30	237.2	1.27
	SpecBundle		450.0	2.37	514.2	2.70	258.6	1.39
Llama-3.3-70B	-	4	540.5	1	575.7	1	512.6	1
	Existing		1272.7	2.35	1049.0	1.82	981.7	1.92
	SpecBundle		1253.0	2.31	1405.1	2.44	1022.7	2.00
Llama-4-Scout	-	8	502.1	1	541.0	1	288.9	1
	Existing		1253.0	2.50	1405.1	2.60	1022.7	3.54
	SpecBundle		1312.4	2.61	1502.2	2.78	1189.6	4.12
Qwen-30B-A3B	-	4	1341.3	1	1410.4	1	1320.1	1
	Existing		2086.1	1.55	2341.3	1.66	1779.0	1.35
	SpecBundle		529.9	1	563.2	1	539.5	1
Qwen-235B-A22B	-	8	642.7	1.21	716.7	1.27	689.4	1.28
	Existing		814.5	1.54	826.5	1.47	889.0	1.65
	SpecBundle		728.5	1	794.1	1	747.7	1
Ling-Flash-V2	-	8	1022.6	1.40	1185.7	1.49	863.9	1.16
	Existing		430.9	1	505.4	1	433.4	1
	SpecBundle		533.8	1.24	811.4	1.61	660.0	1.52
Kimi-K2	-	8	533.8	1.24	811.4	1.61	660.0	1.52
	Existing		533.8	1.24	811.4	1.61	660.0	1.52
	SpecBundle		533.8	1.24	811.4	1.61	660.0	1.52

Table 2. Performance of various models on general benchmarks

nanceQA (Mateega et al., 2025)

We used SGLang as the inference engine to evaluate SpecBundle models on these benchmarks, with all experiments conducted on NVIDIA H200 GPUs. We compared our results against two baselines: (1) standard inference using a single target model and (2) speculative decoding with existing open-source draft models, where available. Several EAGLE3 draft checkpoints were provided by the authors of EAGLE3 (Li et al., 2025) as well as the LMSYS team. Notably, the availability of speculative decoding draft models remains limited, as many target models do not yet have publicly accessible draft checkpoints. For all experiments, we fixed the number of concurrent requests to 8 for LLaMA-3.1-8B due to its smaller model size, and to 16 for larger models, applying tensor parallelism according to the target model scale. We evaluated multiple speculative decoding configurations, varying the number of speculative steps, top- k , and the number of draft tokens, including (3, 1, 4), (5, 1, 6), (5, 3, 6), (7, 1, 8) and (7, 4, 8). We presented

the highest throughput among all configurations.

Table 2 shows the performance on the general benchmarks and Table 3 shows the performance specifically on the coding and math benchmarks. It is evident that SpecBundle significantly outperforms the baselines on all benchmarks and all dense and MoE models: the speedup can reach up to **4.48×** compared to inference with no speculative decoding and **1.35×** compared to inference with an existing draft model.

Particularly for the coding and mathematics benchmarks, SpecBundle achieves speedups over baselines ranging from 1.61× to 4.48× (Table 3). This performance gap arises because existing checkpoints are primarily trained on the ShareGPT and UltraChat datasets, which contain limited coverage of math- and code-centric samples. These results underscore the critical role of data composition in training a well-balanced and high-performing draft model. However, this improvement is not uniform across all domains. A trade-off can be observed, as reflected in the slight decrease in

Target Model	Draft Model	#GPUs	LiveCodeBench		HumanEval		GSM8K		Math500	
			Throughput	Speedup	Throughput	Speedup	Throughput	Speedup	Throughput	Speedup
Llama-3.1-8B	-	1	189.7	1	190.9	1	181.8	1	191.0	1
	Existing		398.4	2.10	480.3	2.52	228.6	1.26	422.4	2.21
	SpecBundle		516.9	2.72	571.5	2.99	329.7	1.81	638.0	3.34
Llama-3.3-70B	-	4	560.9	1	561.0	1	453.2	1	567.4	1
	Existing		1303.4	2.32	1282.8	2.29	521.5	1.15	1122.2	1.98
	SpecBundle		1459.4	2.60	1506.0	2.68	722.0	1.59	1524.9	2.69
Llama-4-Scout	-	8	484.3	1	631.9	1	455.9	1	561.8	1
	Existing		1601.3	3.31	1556.5	2.46	816.6	1.79	1479.0	2.63
	SpecBundle		2170.2	4.48	1944.8	3.08	971.9	2.13	2110.3	3.76
Qwen-30B-A3B	-	4	1492.6	1	1366.6	1	1071.3	1	1469.0	1
	Existing		803.8	1.34	889.9	1.61	697.0	1.49	821.8	1.39
	SpecBundle		1155.7	1.93	1267.5	2.29	758.3	1.62	1399.2	2.38
Qwen-235B-A22B	-	8	598.2	1	553.1	1	469.1	1	587.4	1
	Existing		803.8	1.34	889.9	1.61	697.0	1.49	821.8	1.39
	SpecBundle		1155.7	1.93	1267.5	2.29	758.3	1.62	1399.2	2.38
Ling-Flash-V2	-	8	770.4	1	740.2	1	674.3	1	762.7	1
	Existing		1366.4	1.77	1359.0	1.83	1323.0	1.96	1685.6	2.21
	SpecBundle		1366.4	1.77	1359.0	1.83	1323.0	1.96	1685.6	2.21
Kimi-K2	-	8	500.1	1	466.1	1	337.9	1	492.1	1
	Existing		904.4	1.81	897.9	1.93	544.2	1.61	1022.7	2.08
	SpecBundle		904.4	1.81	897.9	1.93	544.2	1.61	1022.7	2.08

Table 3. Performance of various models on math and coding benchmarks

MT-Bench performance for LLaMA-3 8B and 70B models.

SpecBundle enriches the open-source ecosystem with a broader supply of draft models and delivers substantial performance improvements for production-grade inference. While the current release focuses on instruct models, we plan to extend support to reasoning models and vision-language models in future iterations.

7. Training Insights

We draw some interesting insights for speculative decoding.

7.1. Impact of Data Regeneration

Previous work claims that EAGLE methods exhibit low sensitivity to training data and therefore recommends training directly on the original dataset to reduce computational costs (Li et al., 2024b). However, our empirical results suggest that this assumption does not always hold. We trained an EAGLE3 draft model for LLaMA-3.1-8B using both the original PerfectBlend dataset and a regenerated version. As shown in Figure 5, data regeneration consistently increases the acceptance length across nearly all benchmarks, with FinanceQA being the only exception. Moreover, data regeneration yields an average throughput improvement of 5.3% across all benchmarks.

Although the absolute throughput gain is moderate, data regeneration can have a substantial impact on long-term inference efficiency. Given that speculative decoding is widely deployed in online model serving systems such as the OpenAI API, even modest improvements can translate into significant reductions in inference cost at scale.

7.2. Impact of Training-Time Test

In the original implementation of EAGLE3 (eag, 2025; Li et al., 2025), the TTT length is fixed at 7. We therefore conducted additional experiments to investigate the impact of TTT length on inference performance. Specifically, we varied the TTT length from 1 to 17. As shown in Figure 6, the results indicate that TTT is highly effective in improving the acceptance length, with a sharp gain observed as the TTT length increases from 1 to 3. Moreover, the optimal TTT length is task-dependent. For MT-Bench, a TTT length of 3 already achieves strong performance, whereas for more challenging and longer benchmarks, such as Math500, GSM8K, and HumanEval, a larger TTT length of approximately 13 yields the best results.

However, increasing the TTT length proportionally increases both training time and memory consumption for the draft model, introducing a clear trade-off between performance and efficiency. For domain-specific training, a practical strategy is to first conduct scaling experiments on a small subset of data to identify an appropriate TTT length before training on the full dataset. When training under limited resources, particularly memory constraints, it is advisable to reduce TTT to 3 or 5 to lower memory consumption. For cross-domain training, dynamically adjusting the TTT length based on the sample type could further reduce training cost, as not all samples require the same degree of training-time testing. We leave the design and evaluation of such dynamic TTT strategies to future work.

7.3. Choice of Draft Models

Recently released models such as LLaMA-4, DeepSeek-V3 (DeepSeek-AI et al., 2024), and Kimi-K2 (Team et al., 2025) increasingly adopt the Mixture-of-Experts (MoE) ar-

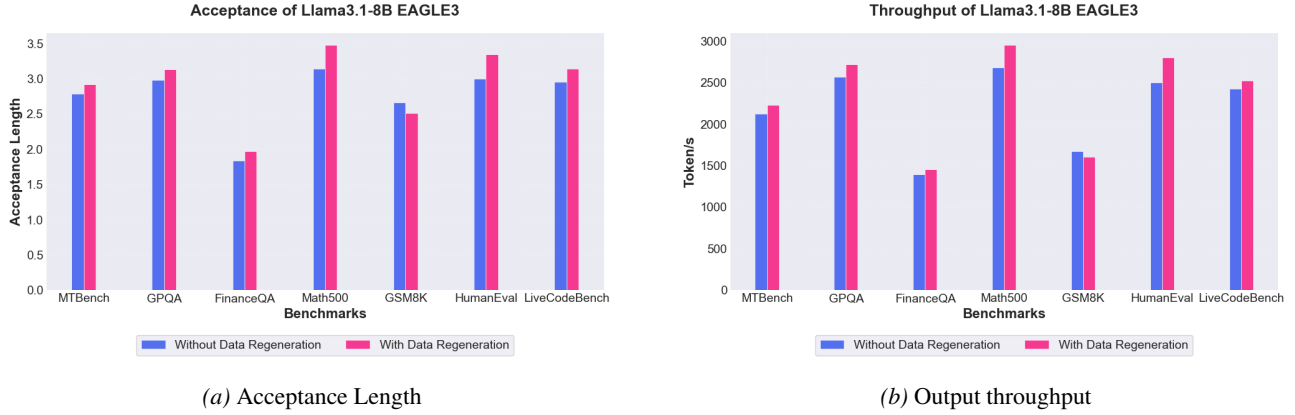


Figure 5. Inference performance of Llama3.1-8B with EAGLE3 trained on datasets with and without regenerating the responses. The experiment was conducted on 1 H200 GPU with batch size 8.

Configuration	#Experts	MTBench	GPQA	FinanceQA	Math500	GSM8K	HumanEval	LiveCodeBench
Same Params	2	1.16	1.05	1.07	1.17	1.05	1.2	1.14
Same FLOPS	2	1.61	1.56	1.34	1.67	1.59	1.64	1.46
	3	1.62	1.69	1.35	1.77	1.7	1.67	1.57
With Shared Experts	4	1.63	1.7	1.36	1.79	1.72	1.69	1.59
	2	1.52	1.69	1.5	1.75	1.61	1.53	1.52
With Shared Experts	3	1.52	1.7	1.46	1.75	1.61	1.53	1.52
	4	1.53	1.69	1.47	1.81	1.67	1.64	1.54
Dense Draft Model	-	2.99	3.14	1.91	2.55	3.48	3.34	3.12

Table 4. Acceptance rate of MoE models with different settings. The results are obtained with configurations MoE top-k = 1, EAGLE3 number of steps = 3, EAGLE3 top-k = 1 and EAGLE3 number of draft tokens = 4.

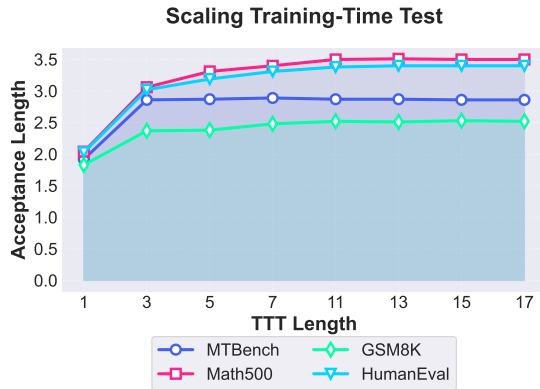


Figure 6. Scaling TTT for the Llama3.1-8B model on the perfect-blend dataset.

chitecture due to its superior performance and inference efficiency. However, existing EAGLE3 draft models remain dense. How to select the architecture of an appropriate draft model remains largely unexplored.

Thus, we conducted experiments to investigate the suitability of MoE models as the draft model. We split the experiments into three categories:

- **Same Parameters:** We initialize two experts in the MoE layer of the draft model, with each expert using an intermediate dimension that is half that of the dense model. As a result, the combined parameter of the MoE layer matches that of the FFN layer in the dense draft model.
- **Same FLOPS:** We construct an MoE layer with two experts, where each expert has the same parameter count as the dense FFN layer. The number of experts selected per token is set to one, ensuring that the total number of floating-point operations remains unchanged.
- **MoE with shared experts:** On top of the "Same FLOPS" setting, we further introduce a shared expert. In this configuration, both the numbers of parameters and the total floating-point operations exceed those of the corresponding dense model.

The results are summarized in Table 4. We observe that the dense draft model consistently outperforms all MoE variants across different settings, indicating that MoE draft models are inherently more difficult to train. Under the Same Params setting, the dense model’s FFN can be viewed as a degenerate two-expert MoE in which one expert has zero parameters. In contrast, the MoE model with the same total parameter budget performs poorly because each expert has fewer parameters than the dense FFN layer and therefore

acts as a weaker learner.

Under the Same FLOPs setting, the MoE draft model performs noticeably better than in the Same Params case, as each expert has increased capacity and can learn more effectively. Nevertheless, its performance still lags behind that of the dense model. This gap arises because the routing top-k is set to 1, meaning that each expert is exposed to fewer tokens during training than the dense FFN, resulting in inferior generalization. Increasing the routing top-k could mitigate this issue, but also proportionally increase the per-token FLOPs, slowing down the drafting process. Consequently, despite their success as target models, MoE architectures are not well suited as draft models for speculative decoding.

8. Conclusion

In this paper, we presented SpecForge, a highly efficient and scalable framework for training speculative decoding draft models, with first-class support for EAGLE3. We introduced target–draft decoupling and a set of optimized kernels that substantially reduce memory consumption and improve training throughput. Extensive experiments demonstrate that SpecForge achieves up to 9.9× speedup over existing approaches. In addition, we released SpecBundle, a collection of production-grade, high-performance EAGLE3 draft models, and conducted systematic training analyses to distill practical insights that facilitate the real-world adoption of speculative decoding.

References

- Eagle-github. <https://github.com/SafeAILab/EAGLE>, 2025.
- Tensorrt-model-optimizer. <https://github.com/NVIDIA/TensorRT-Model-Optimizer/tree/main>, 2025.
- Tensorrt llm. <https://github.com/NVIDIA/TensorRT-LLM>, 2025.
- Achiam, O. J., Adler, S., Agarwal, S., Ahmad, L., Akkaya, I., Aleman, F. L., Almeida, D., Altenschmidt, J., Altman, S., Anadkat, S., Avila, R., Babuschkin, I., Balaji, S., Balcom, V., Baltescu, P., Bao, H., Bavarian, M., et al. Gpt-4 technical report. 2023. URL <https://api.semanticscholar.org/CorpusID:257532815>.
- Agrawal, A., Kedia, N., Panwar, A., Mohan, J., Kwatra, N., Gulavani, B. S., Tumanov, A., and Ramjee, R. Efficient llm inference via chunked prefills. *SIGOPS Oper. Syst. Rev.*, 59(1):9–16, August 2025. ISSN 0163-5980. doi: 10.1145/3759441.3759444. URL <https://doi.org/10.1145/3759441.3759444>.
- Bai, J., Bai, S., Chu, Y., Cui, Z., Dang, K., Deng, X., Fan, Y., Ge, W., Han, Y., Huang, F., Hui, B., Ji, L., Li, M., Lin, J., Lin, R., Liu, D., Liu, G., Lu, C., Lu, K., Ma, J., Men, R., Ren, X., Ren, X., Tan, C., Tan, S., Tu, J., Wang, P., Wang, S., Wang, W., Wu, S., Xu, B., Xu, J., Yang, A., Yang, H., Yang, J., Yang, J., Yang, S., Yao, Y., Yu, B., Bowen, Y., Yuan, H., Yuan, Z., Zhang, J., Zhang, X., Zhang, Y., Zhang, Z., Zhou, C., Zhou, J., Zhou, X., and Zhu, T. Qwen technical report. *ArXiv*, abs/2309.16609, 2023. URL <https://api.semanticscholar.org/CorpusID:263134555>.
- Cai, T., Li, Y., Geng, Z., Peng, H., Lee, J. D., huai Chen, D., and Dao, T. Medusa: Simple llm inference acceleration framework with multiple decoding heads. *ArXiv*, abs/2401.10774, 2024. URL <https://api.semanticscholar.org/CorpusID:267061277>.
- Chen, C., Borgeaud, S., Irving, G., Lespiau, J.-B., Sifre, L., and Jumper, J. M. Accelerating large language model decoding with speculative sampling. *ArXiv*, abs/2302.01318, 2023. URL <https://api.semanticscholar.org/CorpusID:256503945>.
- Chen, J., Feng, A., Zhao, Z., Garza, J., Nurbek, G., Qin, C., Maatouk, A., Tassioulas, L., Gao, Y., and Ying, R. Mtbench: A multimodal time series benchmark for temporal reasoning and question answering. *ArXiv*, abs/2503.16858, 2025. URL <https://api.semanticscholar.org/CorpusID:277244736>.
- Chen, M., Tworek, J., Jun, H., Yuan, Q., de Oliveira Pinto, H. P., Kaplan, J., Edwards, H., Burda, Y., Joseph, N., Brockman, G., Ray, A., Puri, R., Krueger, G., Petrov, M., Khlaaf, H., Sastry, G., Mishkin, P., Chan, B., Gray, S., Ryder, N., Pavlov, M., Power, A., Kaiser, L., Bavarian, M., Winter, C., Tillet, P., Such, F. P., Cummings, D., Plappert, M., Chantzis, F., Barnes, E., Herbert-Voss, A., Guss, W. H., Nichol, A., Paino, A., Tezak, N., Tang, J., Babuschkin, I., Balaji, S., Jain, S., Saunders, W., Hesse, C., Carr, A. N., Leike, J., Achiam, J., Misra, V., Morikawa, E., Radford, A., Knight, M., Brundage, M., Murati, M., Mayer, K., Welinder, P., McGrew, B., Amodei, D., McCandlish, S., Sutskever, I., and Zaremba, W. Evaluating large language models trained on code. 2021.
- Comanici, G., Bieber, E., Schaekermann, M., Pasupat, I., Sachdeva, N., Dhillon, I., Blistein, M., Ram, O., Zhang, D., Rosen, E., et al. Gemini 2.5: Pushing the frontier with advanced reasoning, multimodality, long context, and next generation agentic capabilities. *ArXiv*, abs/2403.05530, 2024. URL <https://api.semanticscholar.org/CorpusID:268297180>.
- DeepSeek-AI, Liu, A., Feng, B., Xue, B., Wang, B.-L., Wu, B., Lu, C., Zhao, C., Deng, C., Zhang, C., Ruan, C., Dai, D., Guo, D., Yang, D., Chen, D., Ji, D.-L., Li, E., Lin, F., Dai, F., Luo, F., Hao, G., et al. Deepseek-v3 technical report. *ArXiv*, abs/2412.19437, 2024. URL <https://api.semanticscholar.org/CorpusID:275118643>.
- DeepSeek-AI, Guo, D., Yang, D., Zhang, H., Song, J.-M., Zhang, R., Xu, R., Zhu, Q., Ma, S., Wang, P., Bi, X., Zhang, X., Yu, X., Wu, Y., Wu, Z. F., Gou, Z., Shao, Z., Li, Z., Gao, Z., Liu, A., Xue, B., et al. Deepseek-r1: Incentivizing reasoning capability in llms via reinforcement learning. *ArXiv*, abs/2501.12948, 2025. URL <https://api.semanticscholar.org/CorpusID:275789950>.
- Ding, N., Chen, Y., Xu, B., Qin, Y., Zheng, Z., Hu, S., Liu, Z., Sun, M., and Zhou, B. Enhancing chat language models by scaling high-quality instructional conversations. *ArXiv*, abs/2305.14233, 2023. URL <https://api.semanticscholar.org/CorpusID:258840897>.
- Du, C., Jiang, J., Xu, Y., Wu, J., Yu, S., Li, Y., Li, S., Xu, K., Nie, L., Tu, Z., and You, Y. Glide with a cape: A low-hassle method to accelerate speculative decoding. *ArXiv*, abs/2402.02082, 2024. URL <https://api.semanticscholar.org/CorpusID:267412316>.
- Dubey, A., Jauhri, A., Pandey, A., Kadian, A., Al-Dahle, A., Letman, A., Mathur, A., Schelten, A., Yang, A., Fan, A., Goyal, A., Hartshorn, A. S., Yang, A., Mitra, A.,

- Sravankumar, A., Korenev, A., et al. The llama 3 herd of models. volume abs/2307.09288, 2023. URL <https://api.semanticscholar.org/CorpusID:259950998>.
- Fu, Y., Bailis, P., Stoica, I., and Zhang, H. Break the sequential dependency of llm inference using lookahead decoding. *ArXiv*, abs/2402.02057, 2024. URL <https://api.semanticscholar.org/CorpusID:267412730>.
- Hu, Y., Liu, Z., Dong, Z., Peng, T., McDanel, B., and Zhang, S. Q. Speculative decoding and beyond: An in-depth survey of techniques. *arXiv preprint arXiv:2502.19732*, 2025.
- Jain, N., Han, K., Gu, A., Li, W.-D., Yan, F., Zhang, T., Wang, S., Solar-Lezama, A., Sen, K., and Stoica, I. Livecodebench: Holistic and contamination free evaluation of large language models for code. *arXiv preprint arXiv:2403.07974*, 2024.
- Kim, S., Mangalam, K., Moon, S., Malik, J., Mahoney, M. W., Gholami, A., and Keutzer, K. **Speculative decoding with big little decoder**. In Oh, A., Naumann, T., Globerson, A., Saenko, K., Hardt, M., and Levine, S. (eds.), *Advances in Neural Information Processing Systems*, volume 36, pp. 39236–39256. Curran Associates, Inc., 2023. URL https://proceedings.neurips.cc/paper_files/paper/2023/file/7b97adeafalc51cf65263459ca9d0d7c-Paper-Conference.pdf.
- Kwon, W., Li, Z., Zhuang, S., Sheng, Y., Zheng, L., Yu, C. H., Gonzalez, J. E., Zhang, H., and Stoica, I. Efficient memory management for large language model serving with pagedattention. In *Proceedings of the ACM SIGOPS 29th Symposium on Operating Systems Principles*, 2023.
- Leviathan, Y., Kalman, M., and Matias, Y. Fast inference from transformers via speculative decoding. In *International Conference on Machine Learning*, 2022. URL <https://api.semanticscholar.org/CorpusID:254096365>.
- Li, Y., Wei, F., Zhang, C., and Zhang, H. Eagle-2: Faster inference of language models with dynamic draft trees. In *Conference on Empirical Methods in Natural Language Processing*, 2024a. URL <https://api.semanticscholar.org/CorpusID:270702281>.
- Li, Y., Wei, F., Zhang, C., and Zhang, H. Eagle: Speculative sampling requires rethinking feature uncertainty. *ArXiv*, abs/2401.15077, 2024b. URL <https://api.semanticscholar.org/CorpusID:267301131>.
- Li, Y., Wei, F., Zhang, C., and Zhang, H. EAGLE-3: Scaling up inference acceleration of large language models via training-time test. In *The Thirty-ninth Annual Conference on Neural Information Processing Systems*, 2025. URL <https://openreview.net/forum?id=4exx1hUffq>.
- Liu, F., Tang, Y., Liu, Z., Ni, Y., Tang, D., Han, K., and Wang, Y. Kangaroo: Lossless self-speculative decoding for accelerating llms via double early exiting. *Advances in Neural Information Processing Systems* 37, 2024. URL <https://api.semanticscholar.org/CorpusID:276117179>.
- Mateega, S., Georgescu, C., and Tang, D. Financeqa: A benchmark for evaluating financial analysis capabilities of large language models, 2025. URL <https://arxiv.org/abs/2501.18062>.
- Miao, X., Oliaro, G., Zhang, Z., Cheng, X., Wang, Z., Zhang, Z., Wong, R. Y. Y., Zhu, A., Yang, L., Shi, X., Shi, C., Chen, Z., Arfeen, D., Abhyankar, R., and Jia, Z. **Specinfer: Accelerating large language model serving with tree-based speculative inference and verification**. In *Proceedings of the 29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 3*, ASPLOS '24, pp. 932–949, New York, NY, USA, 2024. Association for Computing Machinery. ISBN 9798400703867. doi: 10.1145/3620666.3651335. URL <https://doi.org/10.1145/3620666.3651335>.
- Narayanan, D., Shoeybi, M., Casper, J., LeGresley, P., Patwary, M., Korthikanti, V., Vainbrand, D., Kashinkunti, P., Bernauer, J., Catanzaro, B., Phanishayee, A., and Zaharia, M. Efficient large-scale language model training on gpu clusters using megatron-lm. In *Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis*, SC '21, New York, NY, USA, 2021. Association for Computing Machinery. ISBN 9781450384421. doi: 10.1145/3458817.3476209. URL <https://doi.org/10.1145/3458817.3476209>.
- Qwen, :, Yang, A., Yang, B., Zhang, B., Hui, B., Zheng, B., Yu, B., Li, C., Liu, D., Huang, F., Wei, H., Lin, H., Yang, J., Tu, J., Zhang, J., Yang, J., Yang, J., Zhou, J., Lin, J., Dang, K., Lu, K., Bao, K., Yang, K., Yu, L., Li, M., Xue, M., Zhang, P., Zhu, Q., Men, R., Lin, R., Li, T., Tang, T., Xia, T., Ren, X., Ren, X., Fan, Y., Su, Y., Zhang, Y., Wan, Y., Liu, Y., Cui, Z., Zhang, Z., and Qiu, Z. Qwen2.5 technical report, 2025. URL <https://arxiv.org/abs/2412.15115>.
- Raffel, C., Shazeer, N., Roberts, A., Lee, K., Narang, S., Matena, M., Zhou, Y., Li, W., and Liu, P. J. Exploring

- the limits of transfer learning with a unified text-to-text transformer. *J. Mach. Learn. Res.*, 21(1), January 2020. ISSN 1532-4435.
- Rasley, J., Rajbhandari, S., Ruwase, O., and He, Y. Deep-speed: System optimizations enable training deep learning models with over 100 billion parameters. *Proceedings of the 26th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining*, 2020. URL <https://api.semanticscholar.org/CorpusID:221191193>.
- Recasens, P. G., Agullo, F., Zhu, Y., Wang, C., Lee, E. K., Tardieu, O., Torres, J., and Berral, J. L. Mind the memory gap: Unveiling gpu bottlenecks in large-batch llm inference, 2025. URL <https://arxiv.org/abs/2503.08311>.
- Rein, D., Hou, B. L., Stickland, A. C., Petty, J., Pang, R. Y., Dirani, J., Michael, J., and Bowman, S. R. GPQA: A graduate-level google-proof q&a benchmark. In *First Conference on Language Modeling*, 2024. URL <https://openreview.net/forum?id=Ti67584b98>.
- Shah, J., Bikshandi, G., Zhang, Y., Thakkar, V., Ramani, P., and Dao, T. Flashattention-3: fast and accurate attention with asynchrony and low-precision. In *Proceedings of the 38th International Conference on Neural Information Processing Systems*, NIPS '24, Red Hook, NY, USA, 2024. Curran Associates Inc. ISBN 9798331314385.
- Sun, Z., Suresh, A. T., Ro, J. H., Beirami, A., Jain, H., and Yu, F. Spectr: Fast speculative decoding via optimal transport. In Oh, A., Naumann, T., Globerson, A., Saenko, K., Hardt, M., and Levine, S. (eds.), *Advances in Neural Information Processing Systems*, volume 36, pp. 30222–30242. Curran Associates, Inc., 2023. URL https://proceedings.neurips.cc/paper_files/paper/2023/file/6034a661584af6c28fd97a6f23e56c0a-Paper-Conference.pdf.
- Team, K., Bai, Y., Bao, Y., Chen, G., Chen, J., Chen, N., Chen, R., Chen, Y., Chen, Y., Chen, Y., Chen, Z., Cui, J., Ding, H., Dong, M., Du, A., Du, C., Du, D., Du, Y., Fan, Y., Feng, Y., et al. Kimi k2: Open agentic intelligence, 2025. URL <https://arxiv.org/abs/2507.20534>.
- Xia, H., Ge, T., Wang, P., Chen, S.-Q., Wei, F., and Sui, Z. Speculative decoding: Exploiting speculative execution for accelerating seq2seq generation. In *Findings of the Association for Computational Linguistics: EMNLP 2023*, pp. 3909–3925, 2023.
- Xia, H., Li, Y., Zhang, J., Du, C., and Li, W. Swift: On-the-fly self-speculative decoding for llm inference acceleration. *ArXiv*, abs/2410.06916, 2024. URL <https://api.semanticscholar.org/CorpusID:273228257>.
- Xu, T., Helenowski, E., Sankararaman, K. A., Jin, D., Peng, K., Han, E., Nie, S., Zhu, C., Zhang, H., Zhou, W., Zeng, Z., He, Y., Mandyam, K., Talabzadeh, A., Khabsa, M., Cohen, G., Tian, Y., Ma, H., Wang, S., and Fang, H. The perfect blend: Redefining rlhf with mixture of judges, 2024. URL <https://arxiv.org/abs/2409.20370>.
- Yang, A., Li, A., Yang, B., Zhang, B., Hui, B., Zheng, B., Yu, B., Gao, C., Huang, C., Lv, C., Zheng, C., Liu, D., Zhou, F., Huang, F., Hu, F., Ge, H., Wei, H., Lin, H., Tang, J., Yang, J., Tu, J., Zhang, J., Yang, J., Yang, J., Zhou, J., Zhou, J., Lin, J., Dang, K., Bao, K., Yang, K., Yu, L., Deng, L., Li, M., Xue, M., Li, M., Zhang, P., Wang, P., Zhu, Q., Men, R., Gao, R., Liu, S., Luo, S., Li, T., Tang, T., Yin, W., Ren, X., Wang, X., Zhang, X., Ren, X., Fan, Y., Su, Y., Zhang, Y., Zhang, Y., Wan, Y., Liu, Y., Wang, Z., Cui, Z., Zhang, Z., Zhou, Z., and Qiu, Z. Qwen3 technical report, 2025. URL <https://arxiv.org/abs/2505.09388>.
- Ye, Z., Chen, L., Lai, R., Lin, W., Zhang, Y., Wang, S., Chen, T., Kasikci, B., Grover, V., Krishnamurthy, A., and Ceze, L. Flashinfer: Efficient and customizable attention engine for llm inference serving. *ArXiv*, abs/2501.01005, 2025. URL <https://api.semanticscholar.org/CorpusID:275212819>.
- Yu, G.-I. and Jeong, J. S. Orca: A distributed serving system for transformer-based generative models. In *USENIX Symposium on Operating Systems Design and Implementation*, 2022. URL <https://api.semanticscholar.org/CorpusID:251734964>.
- Zhang, J., Wang, J., Li, H., Shou, L., Chen, K., Chen, G., and Mehrotra, S. Draft & verify: Lossless large language model acceleration via self-speculative decoding. In Ku, L.-W., Martins, A., and Srikumar, V. (eds.), *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pp. 11263–11282, Bangkok, Thailand, August 2024a. Association for Computational Linguistics. doi: 10.18653/v1/2024.acl-long.607. URL <https://aclanthology.org/2024.acl-long.607/>.
- Zhang, L., Wang, X., Huang, Y., and Xu, R. Learning harmonized representations for speculative sampling. In *International Conference on Learning Representations*, 2024b. URL <https://api.semanticscholar.org/CorpusID:271974795>.
- Zhang, Y. and Math-AI, T. American invitational mathematics examination (aime) 2024, 2024.

Zhao, Y., Gu, A., Varma, R., Luo, L., chin Huang, C., Xu, M., Wright, L., Shojanazeri, H., Ott, M., Shleifer, S., Desmaison, A., Balioglu, C., Nguyen, B., Chauhan, G., Hao, Y., and Li, S. Pytorch fsdp: Experiences on scaling fully sharded data parallel. *Proc. VLDB Endow.*, 16:3848–3860, 2023. URL <https://api.semanticscholar.org/CorpusID:258297871>.

Zheng, L., Yin, L., Xie, Z., Sun, C., Huang, J., Yu, C. H., Cao, S., Kozyrakis, C., Stoica, I., Gonzalez, J., Barrett, C. W., and Sheng, Y. Sglang: Efficient execution of structured language model programs. *Advances in Neural Information Processing Systems 37*, 2023. URL <https://api.semanticscholar.org/CorpusID:266174771>.